## Exercise 1: Implementing the Singleton Pattern

### Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

### Code:

**SingletonTest.java**

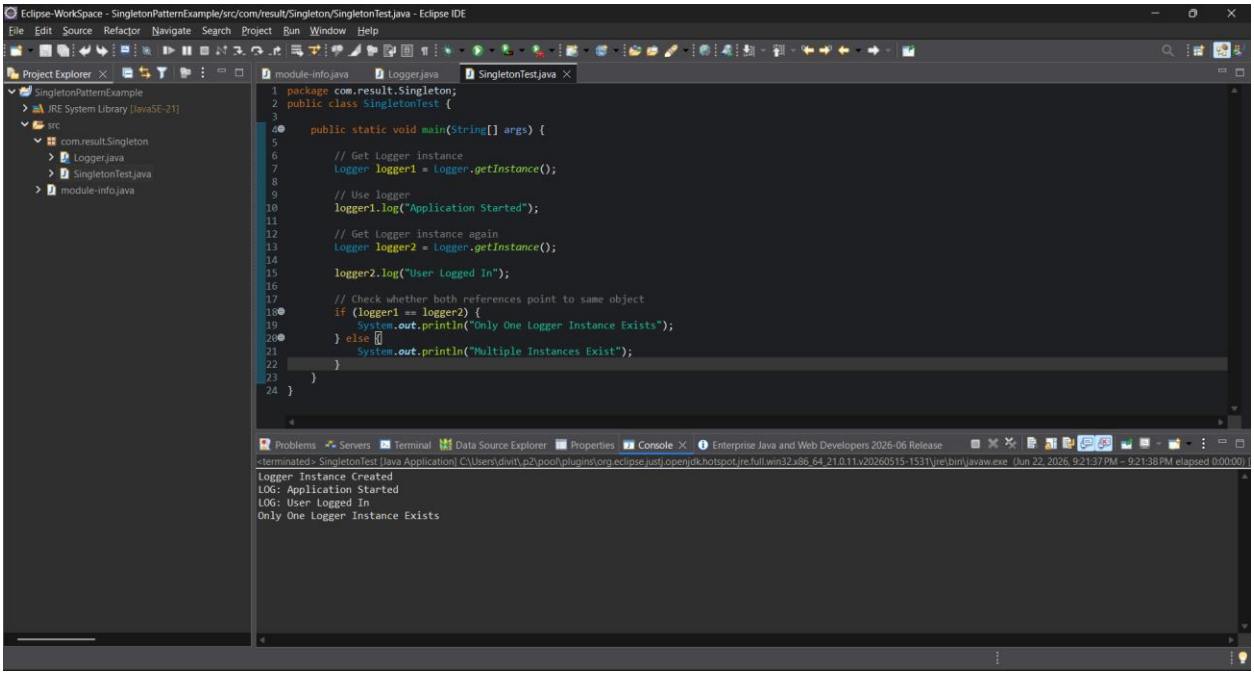
```
package com.result.Singleton;
public class SingletonTest {

    public static void main(String[] args) {

        // Get Logger instance
        Logger logger1 = Logger.getInstance();

        // Use logger
        logger1.log("Application Started");

        // Get Logger instance again
        Logger logger2 = Logger.getInstance();

        logger2.log("User Logged In");

        // Check whether both references point to same object
        if (logger1 == logger2) {
            System.out.println("Only One Logger Instance Exists");
        } else {
            System.out.println("Multiple Instances Exist");
        }
    }
}
```

**Logger.java**

```
package com.result.Singleton;
class Logger {

    // Step 1: Create a private static instance
    private static Logger instance;

    // Step 2: Private constructor
    private Logger() {
        System.out.println("Logger Instance Created");
    }

    // Step 3: Public method to get the instance
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
```

```
        return instance;
    }

    // Logging method
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
```

**Output:**



## Exercise 2: Implementing the Factory Method Pattern

**Scenario:**

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

**Code:**

**Document.java**

```
package com.result.factory;

public interface Document {
    void open();
}
```

**DocumentFactory.java**

```
package com.result.factory;

public abstract class DocumentFactory {

    public abstract Document createDocument();}
```

**ExcelDocument.java**

```
package com.result.factory;

public class ExcelDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Excel Document");
    }
}
```

**ExcelFactory.java**

```
package com.result.factory;

public class ExcelFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new ExcelDocument();
    }
}
```

**PdfDocument.java**

```
package com.result.factory;

public class PdfDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening PDF Document");
    }
}
```

**PdfFactory.java**

```
package com.result.factory;

public class PdfFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new PdfDocument();
    }
}
```

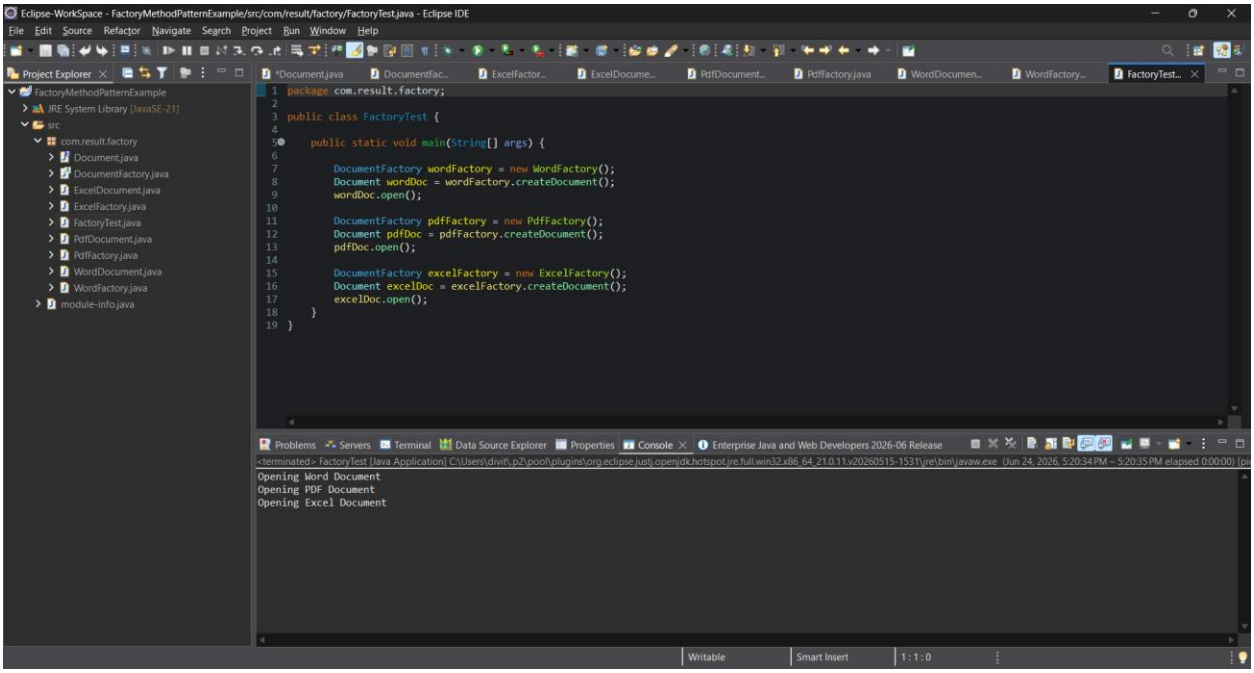
**WordDocument.java**

```
package com.result.factory;

public class WordDocument implements Document {

    @Override
    public void open() {
        System.out.println("Opening Word Document");
    }
}
```

**WordFactory.java**

```
package com.result.factory;

public class WordFactory extends DocumentFactory {

    @Override
    public Document createDocument() {
        return new WordDocument();
    }
}
```

**Output:**


Eclipse-WorkSpace - FactoryMethodPatternExample/src/com/result/factory/FactoryTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
FactoryMethodPatternExample
JRE System Library [JavaSE-21]
src
com.result.factory
Document.java
DocumentFactory.java
ExcelDocument.java
ExcelFactory.java
FactoryTest.java
PdfDocument.java
PdfFactory.java
WordDocument.java
WordFactory.java
module-info.java
*Document.java
DocumentFac...
ExcelFactor...
ExcelDocume...
PdfDocument...
PdfFactory.java
WordDocumen...
WordFactory...
FactoryTest...
1 package com.result.factory;
2
3 public class FactoryTest {
4
5 public static void main(String[] args) {
6
7 DocumentFactory wordFactory = new WordFactory();
8 Document wordDoc = wordFactory.createDocument();
9 wordDoc.open();
10
11 DocumentFactory pdfFactory = new PdfFactory();
12 Document pdfDoc = pdfFactory.createDocument();
13 pdfDoc.open();
14
15 DocumentFactory excelFactory = new ExcelFactory();
16 Document excelDoc = excelFactory.createDocument();
17 excelDoc.open();
18 }
19 }
Problems Servers Terminal Data Source Explorer Properties Console Enterprise Java and Web Developers 2026-06 Release
<terminated> FactoryTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 24, 2026, 5:20:34 PM – 5:20:35 PM elapsed 0:00:00)
Opening Word Document
Opening PDF Document
Opening Excel Document
Writable
Smart Insert
1 : 1 : 0

## Exercise 3: Implementing the Builder Pattern

### Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

**Code:**

**Computer.java**

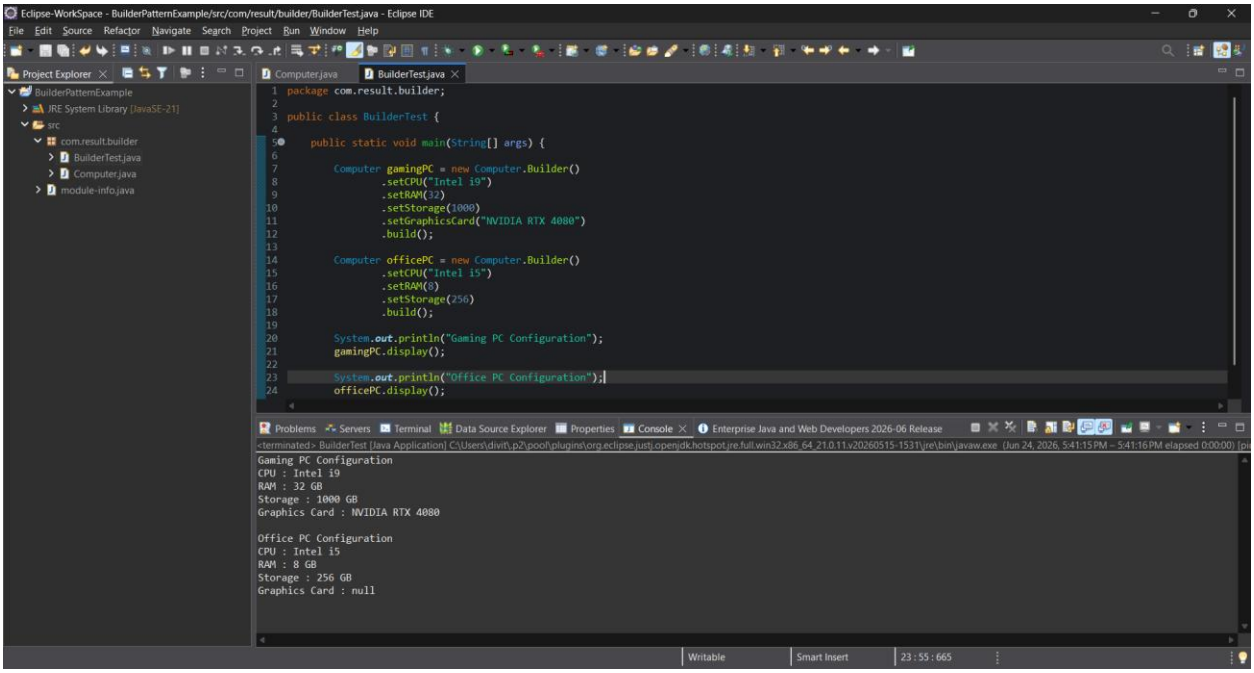
```
package com.result.builder;

public class Computer {

    private String cpu;
    private int ram;
    private int storage;
    private String graphicsCard;

    // Private Constructor
    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
        this.graphicsCard = builder.graphicsCard;
    }

    public void display() {
        System.out.println("CPU : " + cpu);
        System.out.println("RAM : " + ram + " GB");
        System.out.println("Storage : " + storage + " GB");
        System.out.println("Graphics Card : " + graphicsCard);
        System.out.println();
    }

    // Static Nested Builder Class
    public static class Builder {

        private String cpu;
        private int ram;
        private int storage;
        private String graphicsCard;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(int storage) {
```

```
            this.storage = storage;
            return this;
        }

        public Builder setGraphicsCard(String graphicsCard) {
            this.graphicsCard = graphicsCard;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

**BuilderTest.java**

```
package com.result.builder;

public class BuilderTest {

    public static void main(String[] args) {

        Computer gamingPC = new Computer.Builder()
                .setCPU("Intel i9")
                .setRAM(32)
                .setStorage(1000)
                .setGraphicsCard("NVIDIA RTX 4080")
                .build();

        Computer officePC = new Computer.Builder()
                .setCPU("Intel i5")
                .setRAM(8)
                .setStorage(256)
                .build();

        System.out.println("Gaming PC Configuration");
        gamingPC.display();

        System.out.println("Office PC Configuration");
        officePC.display();
    }
}
```

**Output:**


Eclipse-WorkSpace - BuilderPatternExample/src/com/result/builder/BuilderTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
BuilderPatternExample
JRE System Library [JavaSE-21]
src
com.result.builder
BuilderTest.java
Computer.java
module-info.java
Computer.java
BuilderTest.java
package com.result.builder;
public class BuilderTest {
public static void main(String[] args) {
Computer gamingPC = new Computer.Builder()
.setCPU("Intel i9")
.setRAM(32)
.setStorage(1000)
.setGraphicsCard("NVIDIA RTX 4080")
.build();
Computer officePC = new Computer.Builder()
.setCPU("Intel i5")
.setRAM(8)
.setStorage(256)
.build();
System.out.println("Gaming PC Configuration");
gamingPC.display();
System.out.println("Office PC Configuration");
officePC.display();
Problems Servers Terminal Data Source Explorer Properties Console Enterprise Java and Web Developers 2026-06 Release
<terminated> BuilderTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 24, 2026, 5:41:15 PM – 5:41:16 PM elapsed 0:00:00)
Gaming PC Configuration
CPU : Intel i9
RAM : 32 GB
Storage : 1000 GB
Graphics Card : NVIDIA RTX 4080
Office PC Configuration
CPU : Intel i5
RAM : 8 GB
Storage : 256 GB
Graphics Card : null
Writable
Smart Insert
23 : 55 : 665

## Exercise 4: Implementing the Adapter Pattern

### Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

**Code:**

**PaymentProcessor.java**

```
package com.result.adapter;

public interface PaymentProcessor {
    void processPayment(double amount);
}
```

**PayPalAdapter.java**

```
package com.result.adapter;

public class PayPalAdapter implements PaymentProcessor {

    private PayPalGateway payPalGateway;

    public PayPalAdapter() {
        payPalGateway = new PayPalGateway();
    }

    @Override
    public void processPayment(double amount) {
        payPalGateway.makePayment(amount);
    }
}
```

**RazorpayAdapter.java**

```
package com.result.adapter;

public class RazorpayAdapter implements PaymentProcessor {

    private RazorpayGateway razorpayGateway;

    public RazorpayAdapter() {
        razorpayGateway = new RazorpayGateway();
    }

    @Override
    public void processPayment(double amount) {
        razorpayGateway.payAmount(amount);
    }
}
```

**StripeAdapter.java**

```java
package com.result.adapter;

public class StripeAdapter implements PaymentProcessor {

    private StripeGateway stripeGateway;

    public StripeAdapter() {
        stripeGateway = new StripeGateway();
    }

    @Override
    public void processPayment(double amount) {
        stripeGateway.sendMoney(amount);
    }
}
```

**PayPalGateway.java**

```java
package com.result.adapter;

public class PayPalGateway {

    public void makePayment(double amount) {
        System.out.println("Payment of Rs." + amount + " processed using PayPal");
    }
}
```

**StripeGateway.java**

```java
package com.result.adapter;

public class StripeGateway {

    public void sendMoney(double amount) {
        System.out.println("Payment of Rs." + amount + " processed using Stripe");
    }
}
```

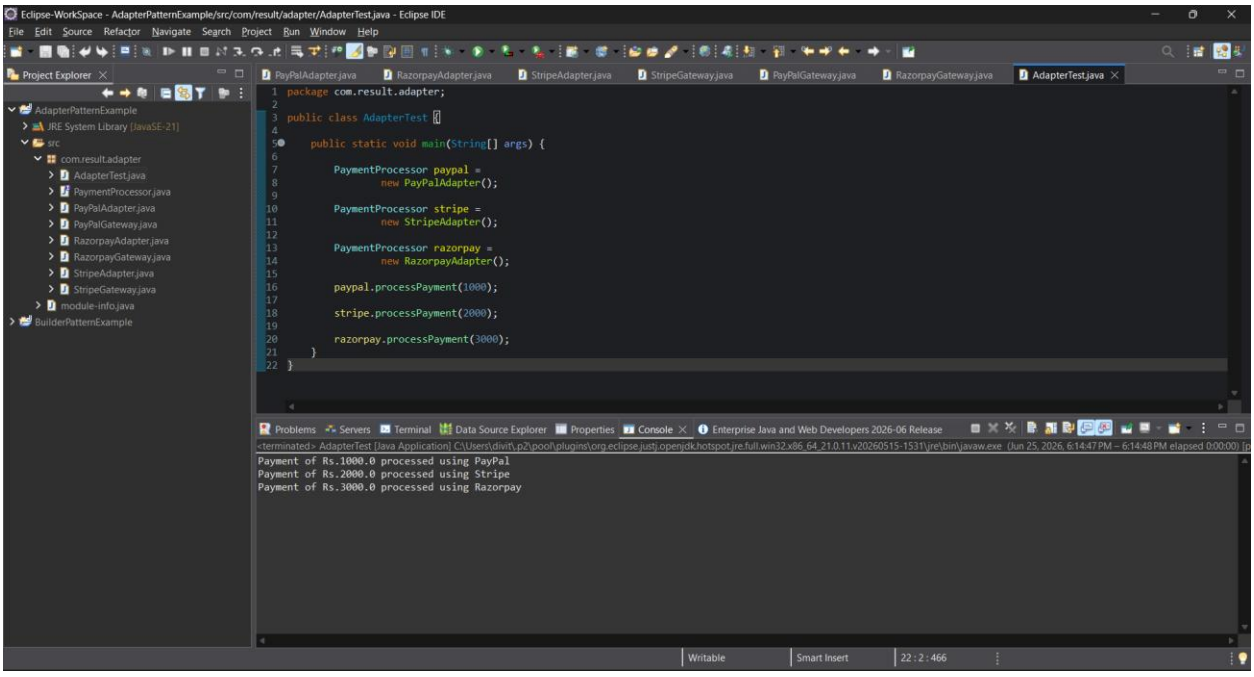
**RazorpayGateway.java**

```java
package com.result.adapter;

public class RazorpayGateway {

    public void payAmount(double amount) {
        System.out.println("Payment of Rs." + amount + " processed using Razorpay");
    }
}
```

**AdapterTest.java**

```java
package com.result.adapter;

public class AdapterTest {

    public static void main(String[] args) {

        PaymentProcessor paypal =
                new PayPalAdapter();

        PaymentProcessor stripe =
                new StripeAdapter();

        PaymentProcessor razorpay =
                new RazorpayAdapter();

        paypal.processPayment(1000);

        stripe.processPayment(2000);

        razorpay.processPayment(3000);
    }
}
```

**Output:**



## Exercise 5: Implementing the Decorator Pattern

### Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

**Code:**

**Notifier.java**

```
package com.result.decorator;

public interface Notifier {
    void send(String message);
}
```

**EmailNotifier.java**

```
package com.result.decorator;

public class EmailNotifier implements Notifier {

    @Override
    public void send(String message) {
        System.out.println("Email Notification: " + message);
    }
}
```

**NotifierDecorator.java**

```
package com.result.decorator;

public abstract class NotifierDecorator implements Notifier {

    protected Notifier notifier;

    public NotifierDecorator(Notifier notifier) {
        this.notifier = notifier;
    }

    @Override
    public void send(String message) {
        notifier.send(message);
    }
}
```

**SMSNotifierDecorator.java**

```java
package com.result.decorator;

public class SMSNotifierDecorator extends NotifierDecorator {

    public SMSNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        super.send(message);
        sendSMS(message);
    }

    private void sendSMS(String message) {
        System.out.println("SMS Notification: " + message);
    }
}
```

**SlackNotifierDecorator.java**

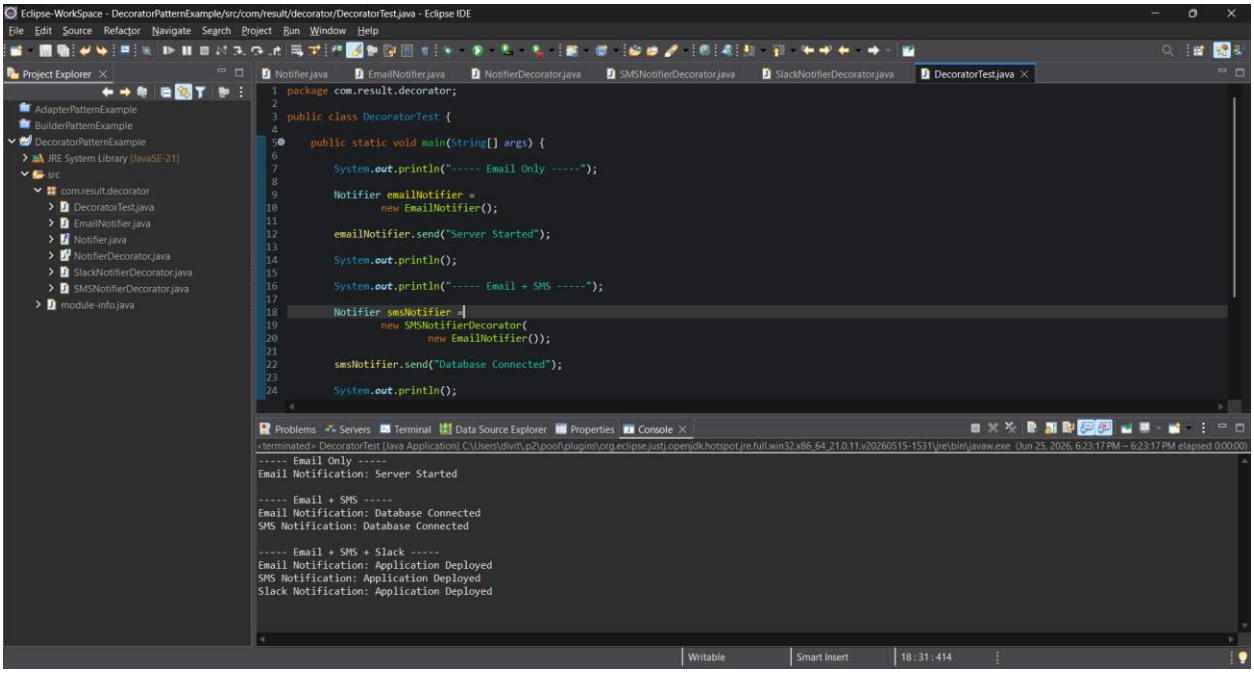
```java
package com.result.decorator;

public class SlackNotifierDecorator extends NotifierDecorator {

    public SlackNotifierDecorator(Notifier notifier) {
        super(notifier);
    }

    @Override
    public void send(String message) {
        super.send(message);
        sendSlack(message);
    }

    private void sendSlack(String message) {
        System.out.println("Slack Notification: " + message);
    }
}
```

**DecoratorTest.java**

```java
package com.result.decorator;

public class DecoratorTest {

    public static void main(String[] args) {

        System.out.println("----- Email Only -----");

        Notifier emailNotifier =
```

```
                new EmailNotifier();

        emailNotifier.send("Server Started");

        System.out.println();

        System.out.println("----- Email + SMS -----");

        Notifier smsNotifier =
                new SMSNotifierDecorator(
                        new EmailNotifier());

        smsNotifier.send("Database Connected");

        System.out.println();

        System.out.println("----- Email + SMS + Slack -----");

        Notifier fullNotifier =
                new SlackNotifierDecorator(
                        new SMSNotifierDecorator(
                                new EmailNotifier()));

        fullNotifier.send("Application Deployed");
    }
}
```

**Output:**



## Exercise 6: Implementing the Proxy Pattern

### Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

**Output:**

**Image.java**

```
package com.result.proxy;

public interface Image {
    void display();
}
```

**ProxyImage.java**

```
package com.result.proxy;

public class ProxyImage implements Image {

    private RealImage realImage;
    private String fileName;

    public ProxyImage(String fileName) {
        this.fileName = fileName;
    }

    @Override
    public void display() {

        // Lazy Initialization
        if (realImage == null) {
            realImage = new RealImage(fileName);
        }

        // Cached object is reused
        realImage.display();
    }
}
```

**RealImage.java**

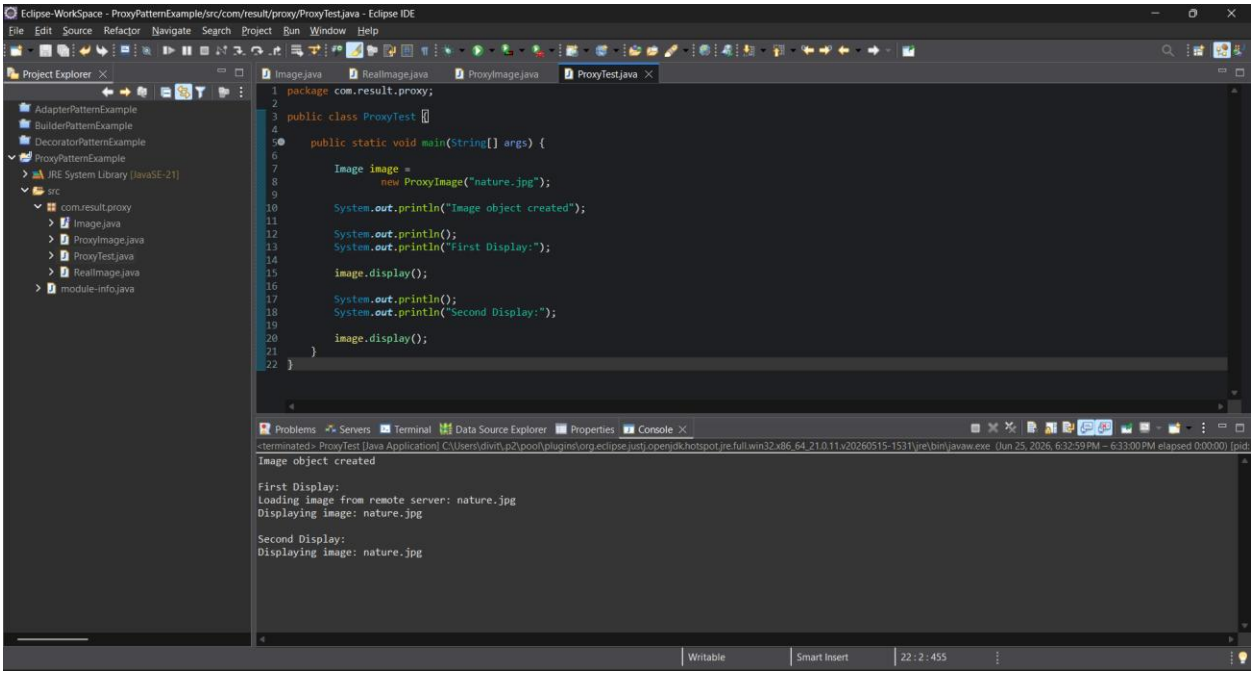
```
package com.result.proxy;

public class RealImage implements Image {

    private String fileName;

    public RealImage(String fileName) {
        this.fileName = fileName;
```

```
        loadFromServer();
    }

    private void loadFromServer() {
        System.out.println("Loading image from remote server: "
                + fileName);
    }

    @Override
    public void display() {
        System.out.println("Displaying image: "
                + fileName);
    }
}
```

**ProxyTest.java**

```
package com.result.proxy;

public class ProxyTest {

    public static void main(String[] args) {

        Image image =
                new ProxyImage("nature.jpg");

        System.out.println("Image object created");

        System.out.println();
        System.out.println("First Display:");

        image.display();

        System.out.println();
        System.out.println("Second Display:");

        image.display();
    }
}
```

**Output:**


Eclipse-WorkSpace - ProxyPatternExample/src/com/result/proxy/ProxyTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
AdapterPatternExample
BuilderPatternExample
DecoratorPatternExample
ProxyPatternExample
JRE System Library [JavaSE-21]
src
com.result.proxy
Image.java
ProxyImage.java
ProxyTest.java
RealImage.java
module-info.java
Image.java
RealImage.java
ProxyImage.java
ProxyTest.java
1 package com.result.proxy;
2
3 public class ProxyTest {
4
5 public static void main(String[] args) {
6
7 Image image =
8 new ProxyImage("nature.jpg");
9
10 System.out.println("Image object created");
11
12 System.out.println();
13 System.out.println("First Display:");
14
15 image.display();
16
17 System.out.println();
18 System.out.println("Second Display:");
19
20 image.display();
21 }
22 }
Problems Servers Terminal Data Source Explorer Properties Console
<terminated> ProxyTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 25, 2026, 6:32:59 PM – 6:33:00 PM elapsed 0:00:00) [pid:
Image object created
First Display:
Loading image from remote server: nature.jpg
Displaying image: nature.jpg
Second Display:
Displaying image: nature.jpg
Writable
Smart Insert
22 : 2 : 455

## Exercise 7: Implementing the Observer Pattern

### Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

**Code:**

**Stock.java**

```
package com.result.observer;

public interface Stock {

    void registerObserver(Observer observer);

    void removeObserver(Observer observer);

    void notifyObservers();
}
```

**Observer.java**

```
package com.result.observer;

public interface Observer {

    void update(String stockName, double price);
}
```

**StockMarket.java**

```
package com.result.observer;

import java.util.ArrayList;
import java.util.List;

public class StockMarket implements Stock {

    private List<Observer> observers =
            new ArrayList<>();

    private String stockName;
    private double price;

    @Override
    public void registerObserver(
            Observer observer) {

        observers.add(observer);
    }
```

```
    @Override
    public void removeObserver(
            Observer observer) {

        observers.remove(observer);
    }

    @Override
    public void notifyObservers() {

        for (Observer observer : observers) {
            observer.update(stockName, price);
        }
    }

    public void setStockPrice(
            String stockName,
            double price) {

        this.stockName = stockName;
        this.price = price;

        System.out.println(
                "\nStock Updated -> "
                        + stockName
                        + " : Rs."
                        + price);

        notifyObservers();
    }
}
```

**MobileApp.java**

```
package com.result.observer;

public class MobileApp implements Observer {

    @Override
    public void update(
            String stockName,
            double price) {

        System.out.println(
                "Mobile App Notification -> "
                        + stockName
                        + " = Rs."
                        + price);
    }
}
```

**WebApp.java**

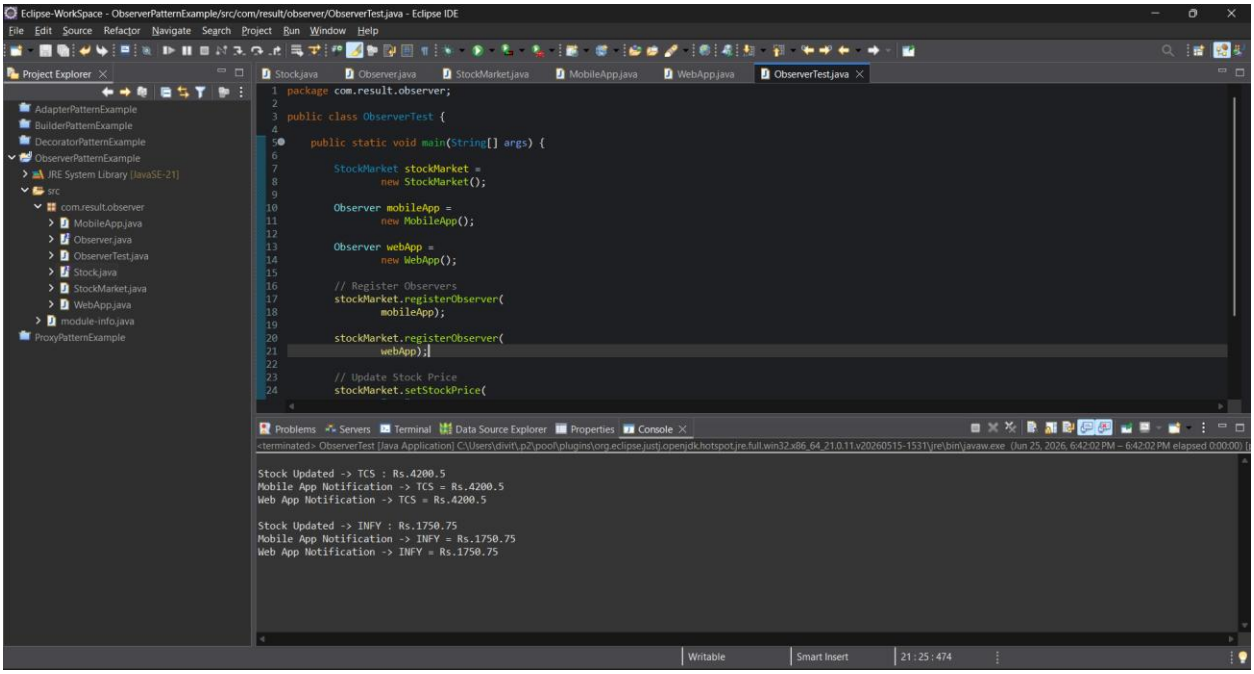
```
package com.result.observer;

public class WebApp implements Observer {

    @Override
    public void update(
            String stockName,
            double price) {

        System.out.println(
                "Web App Notification -> "
                        + stockName
                        + " = Rs."
                        + price);
    }
}
```

**ObserverTest.java**

```
package com.result.observer;

public class ObserverTest {

    public static void main(String[] args) {

        StockMarket stockMarket =
                new StockMarket();

        Observer mobileApp =
                new MobileApp();

        Observer webApp =
                new WebApp();

        // Register Observers
        stockMarket.registerObserver(
                mobileApp);

        stockMarket.registerObserver(
                webApp);

        // Update Stock Price
        stockMarket.setStockPrice(
                "TCS",
                4200.50);

        stockMarket.setStockPrice(
                "INFY",
                1750.75);
    }
```

```
}
```

## Output:


Eclipse-WorkSpace - ObserverPatternExample/src/com/result/observer/ObserverTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
AdapterPatternExample
BuilderPatternExample
DecoratorPatternExample
ObserverPatternExample
JRE System Library [JavaSE-21]
src
com.result.observer
MobileApp.java
Observer.java
ObserverTest.java
Stock.java
StockMarket.java
WebApp.java
module-info.java
ProxyPatternExample
Stock.java
Observer.java
StockMarket.java
MobileApp.java
WebApp.java
ObserverTest.java
package com.result.observer;
public class ObserverTest {
public static void main(String[] args) {
StockMarket stockMarket =
new StockMarket();
Observer mobileApp =
new MobileApp();
Observer webApp =
new WebApp();
// Register Observers
stockMarket.registerObserver(
mobileApp);
stockMarket.registerObserver(
webApp);
// Update Stock Price
stockMarket.setStockPrice(
Problems Servers Terminal Data Source Explorer Properties Console
<terminated> ObserverTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 25, 2026, 6:42:02 PM – 6:42:02 PM elapsed 0:00:00)
Stock Updated -> TCS : Rs.4200.5
Mobile App Notification -> TCS = Rs.4200.5
Web App Notification -> TCS = Rs.4200.5
Stock Updated -> INFY : Rs.1750.75
Mobile App Notification -> INFY = Rs.1750.75
Web App Notification -> INFY = Rs.1750.75
Writable
Smart Insert
21 : 25 : 474

## Exercise 8: Implementing the Strategy Pattern

### Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

**Code:**

**PaymentStrategy.java**

```
package com.result.strategy;

public interface PaymentStrategy {

    void pay(double amount);
}
```

**CreditCardPayment.java**

```
package com.result.strategy;

public class CreditCardPayment
        implements PaymentStrategy {

    @Override
    public void pay(double amount) {

        System.out.println(
                "Paid Rs."
                        + amount
                        + " using Credit Card");
    }
}
```

**PayPalPayment.java**

```
package com.result.strategy;

public class PayPalPayment
        implements PaymentStrategy {

    @Override
    public void pay(double amount) {

        System.out.println(
                "Paid Rs."
                        + amount
                        + " using PayPal");
    }
}
```

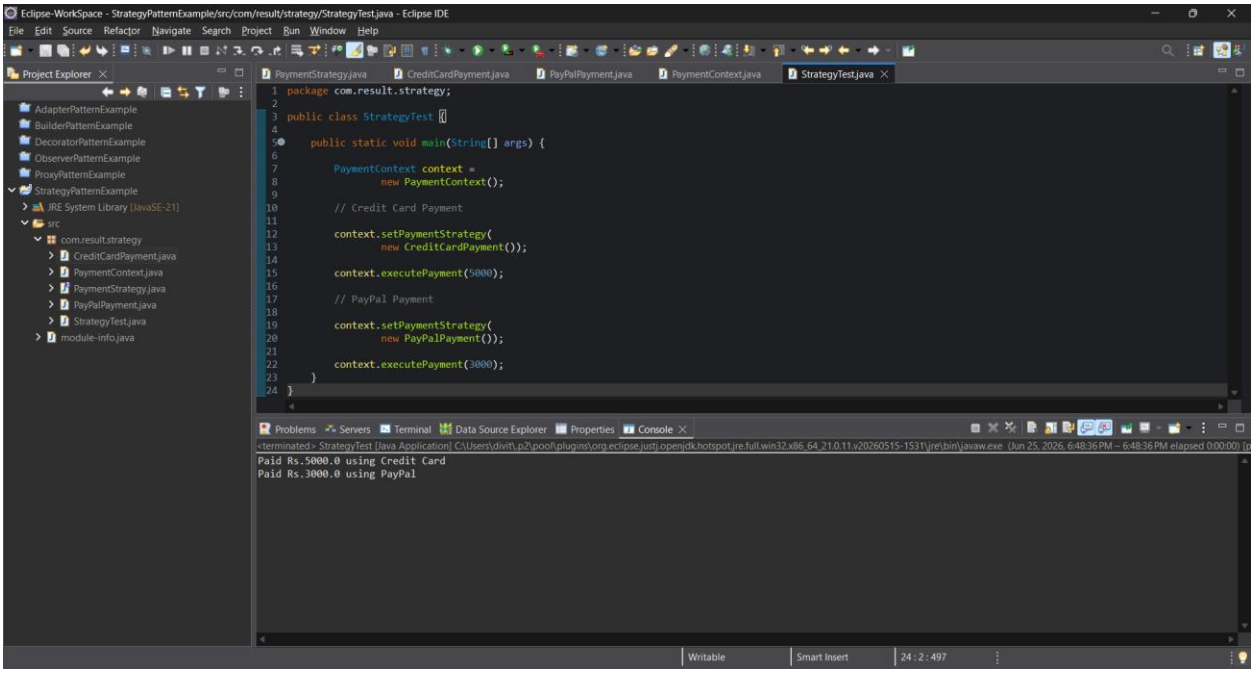
**PaymentContext.java**

```
package com.result.strategy;

public class PaymentContext {

    private PaymentStrategy paymentStrategy;

    public void setPaymentStrategy(
            PaymentStrategy paymentStrategy) {

        this.paymentStrategy =
                paymentStrategy;
    }

    public void executePayment(
            double amount) {

        paymentStrategy.pay(amount);
    }
}
```

**StrategyTest.java**

```
package com.result.strategy;

public class StrategyTest {

    public static void main(String[] args) {

        PaymentContext context =
                new PaymentContext();

        // Credit Card Payment

        context.setPaymentStrategy(
                new CreditCardPayment());

        context.executePayment(5000);

        // PayPal Payment

        context.setPaymentStrategy(
                new PayPalPayment());

        context.executePayment(3000);
    }
}
```

**Output:**


Eclipse-WorkSpace - StrategyPatternExample/src/com/result/strategy/StrategyTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
AdapterPatternExample
BuilderPatternExample
DecoratorPatternExample
ObserverPatternExample
ProxyPatternExample
StrategyPatternExample
JRE System Library [JavaSE-21]
src
com.result.strategy
CreditCardPayment.java
PaymentContext.java
PaymentStrategy.java
PayPalPayment.java
StrategyTest.java
module-info.java
PaymentStrategy.java
CreditCardPayment.java
PayPalPayment.java
PaymentContext.java
StrategyTest.java
package com.result.strategy;
public class StrategyTest {
public static void main(String[] args) {
PaymentContext context =
new PaymentContext();
// Credit Card Payment
context.setPaymentStrategy(
new CreditCardPayment());
context.executePayment(5000);
// PayPal Payment
context.setPaymentStrategy(
new PayPalPayment());
context.executePayment(3000);
}
}
Problems Servers Terminal Data Source Explorer Properties Console
<terminated> StrategyTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 25, 2026, 6:48:36 PM – 6:48:36 PM elapsed 0:00:00)
Paid Rs.5000.0 using Credit Card
Paid Rs.3000.0 using PayPal
Writable
Smart Insert
24 : 2 : 497

## Exercise 9: Implementing the Command Pattern

**Scenario:** You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

**Code:**

**Command.java**

```
package com.result.command;

public interface Command {

    void execute();
}
```

**Light.java**

```
package com.result.command;

public class Light {

    public void turnOn() {
        System.out.println("Light is ON");
    }

    public void turnOff() {
        System.out.println("Light is OFF");
    }
}
```

**LightOnCommand.java**

```
package com.result.command;

public class LightOnCommand
        implements Command {

    private Light light;

    public LightOnCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOn();
    }
}
```

**LightOffCommand.java**

```
package com.result.command;

public class LightOffCommand
        implements Command {
```

```
    private Light light;

    public LightOffCommand(Light light) {
        this.light = light;
    }

    @Override
    public void execute() {
        light.turnOff();
    }
}
```

**RemoteControl.java**

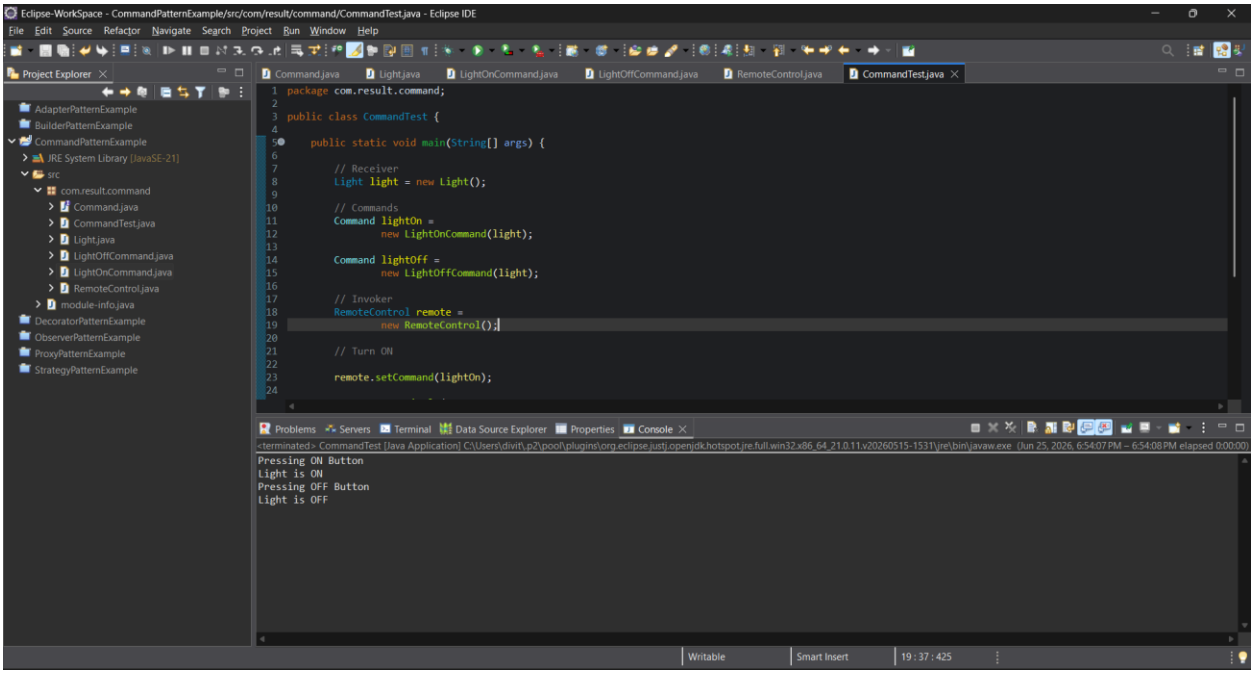
```
package com.result.command;

public class RemoteControl {

    private Command command;

    public void setCommand(
            Command command) {

        this.command = command;
    }

    public void pressButton() {

        command.execute();
    }
}
```

**CommandTest.java**

```
package com.result.command;

public class CommandTest {

    public static void main(String[] args) {

        // Receiver
        Light light = new Light();

        // Commands
        Command lightOn =
                new LightOnCommand(light);

        Command lightOff =
                new LightOffCommand(light);

        // Invoker
        RemoteControl remote =
                new RemoteControl();

        // Turn ON
```

```
        remote.setCommand(lightOn);

        System.out.println(
                "Pressing ON Button");

        remote.pressButton();

        // Turn OFF

        remote.setCommand(lightOff);

        System.out.println(
                "Pressing OFF Button");

        remote.pressButton();
    }
}
```

**Output:**



## Exercise 10: Implementing the MVC Pattern

### Scenario:

You are developing a simple web application for managing student records using the MVC pattern.

**Code:**

**Student.java**

```
package com.result.mvc;

public class Student {

    private String name;
    private int id;
    private String grade;

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getGrade() {
        return grade;
    }

    public void setGrade(String grade) {
        this.grade = grade;
    }
}
```

**StudentView.java**

```
package com.result.mvc;

public class StudentView {

    public void displayStudentDetails(
            String studentName,
            int studentId,
            String studentGrade) {
```

```
        System.out.println("Student Details");
        System.out.println("--------------------");
        System.out.println("Name  : " + studentName);
        System.out.println("ID    : " + studentId);
        System.out.println("Grade : " + studentGrade);
    }
}
```

**StudentController.java**

```
package com.result.mvc;

public class StudentController {

    private Student model;
    private StudentView view;

    public StudentController(
            Student model,
            StudentView view) {

        this.model = model;
        this.view = view;
    }

    public void setStudentName(
            String name) {

        model.setName(name);
    }

    public String getStudentName() {
        return model.getName();
    }

    public void setStudentId(
            int id) {

        model.setId(id);
    }

    public int getStudentId() {
        return model.getId();
    }

    public void setStudentGrade(
            String grade) {

        model.setGrade(grade);
    }

    public String getStudentGrade() {
        return model.getGrade();
    }
```

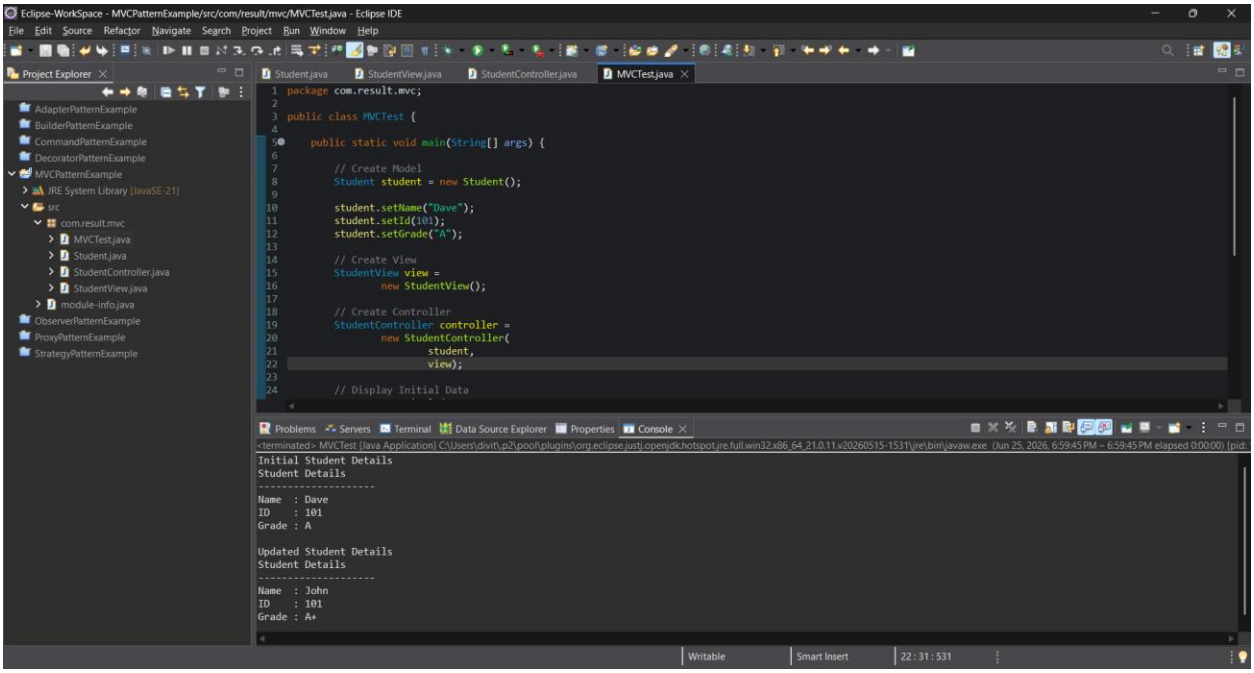
```
    public void updateView() {

        view.displayStudentDetails(
                model.getName(),
                model.getId(),
                model.getGrade());
    }
}
```

**MVCTest.java**

```
package com.result.mvc;

public class MVCTest {

    public static void main(String[] args) {

        // Create Model
        Student student = new Student();

        student.setName("Dave");
        student.setId(101);
        student.setGrade("A");

        // Create View
        StudentView view =
                new StudentView();

        // Create Controller
        StudentController controller =
                new StudentController(
                        student,
                        view);

        // Display Initial Data
        System.out.println(
                "Initial Student Details");

        controller.updateView();

        System.out.println();

        // Update Data Through Controller
        controller.setStudentName(
                "John");

        controller.setStudentGrade(
                "A+");

        System.out.println(
                "Updated Student Details");

        controller.updateView();
```

```
    }
}
```

**Output:**


Eclipse-WorkSpace - MVCPatternExample/src/com/result/mvc/MVCTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
AdapterPatternExample
BuilderPatternExample
CommandPatternExample
DecoratorPatternExample
MVCPatternExample
JRE System Library [JavaSE-21]
src
com.result.mvc
MVCTest.java
Student.java
StudentController.java
StudentView.java
module-info.java
ObserverPatternExample
ProxyPatternExample
StrategyPatternExample
Student.java
StudentView.java
StudentController.java
MVCTest.java
package com.result.mvc;
public class MVCTest {
public static void main(String[] args) {
// Create Model
Student student = new Student();
student.setName("Dave");
student.setId(101);
student.setGrade("A");
// Create View
StudentView view =
new StudentView();
// Create Controller
StudentController controller =
new StudentController(
student,
view);
// Display Initial Data
Problems Servers Terminal Data Source Explorer Properties Console
<terminated> MVCTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 25, 2026, 6:59:45 PM – 6:59:45 PM elapsed 0:00:00) [pid:
Initial Student Details
Student Details
Name : Dave
ID : 101
Grade : A
Updated Student Details
Student Details
Name : John
ID : 101
Grade : A+
Writable
Smart Insert
22 : 31 : 531

## Exercise 11: Implementing Dependency Injection

### Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

**Code:**

**CustomerRepository.java**

```
package com.result.di;

public interface CustomerRepository {

    String findCustomerById(int id);
}
```

**CustomerRepositoryImpl.java**

```
package com.result.di;

public class CustomerRepositoryImpl
        implements CustomerRepository {

    @Override
    public String findCustomerById(int id) {

        if (id == 101) {
            return "Divith";
        }

        return "Customer Not Found";
    }
}
```

**CustomerService.java**

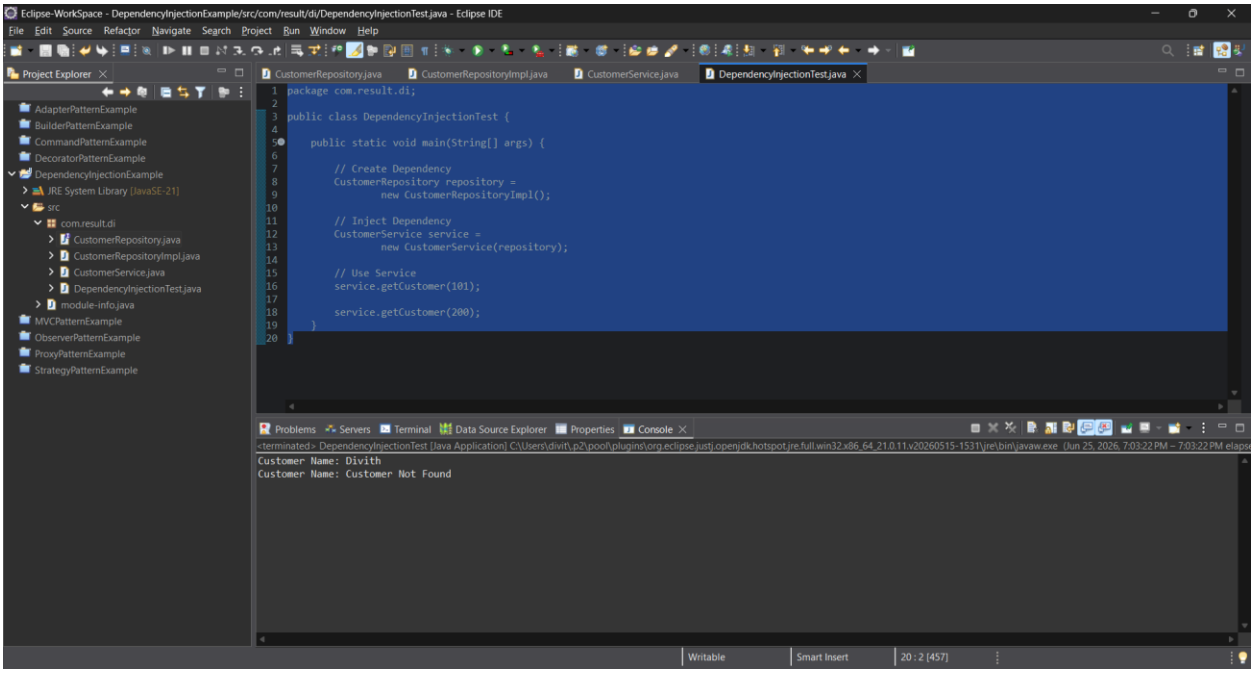
```
package com.result.di;

public class CustomerService {

    private CustomerRepository repository;

    // Constructor Injection
```

```
    public CustomerService(
            CustomerRepository repository) {

        this.repository = repository;
    }

    public void getCustomer(int id) {

        String customer =
                repository.findCustomerById(id);

        System.out.println(
                "Customer Name: " + customer);
    }
}
```

**DependencyInjectionTest.java**

```
package com.result.di;

public class DependencyInjectionTest {

    public static void main(String[] args) {

        // Create Dependency
        CustomerRepository repository =
                new CustomerRepositoryImpl();

        // Inject Dependency
        CustomerService service =
                new CustomerService(repository);

        // Use Service
        service.getCustomer(101);

        service.getCustomer(200);
    }
}
```

**Output:**


Eclipse-WorkSpace - DependencyInjectionExample/src/com/result/di/DependencyInjectionTest.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help
Project Explorer
AdapterPatternExample
BuilderPatternExample
CommandPatternExample
DecoratorPatternExample
DependencyInjectionExample
JRE System Library [JavaSE-21]
src
com.result.di
CustomerRepository.java
CustomerRepositoryImpl.java
CustomerService.java
DependencyInjectionTest.java
module-info.java
MVCPatternExample
ObserverPatternExample
ProxyPatternExample
StrategyPatternExample
CustomerRepository.java
CustomerRepositoryImpl.java
CustomerService.java
DependencyInjectionTest.java
1 package com.result.di;
2
3 public class DependencyInjectionTest {
4
5 public static void main(String[] args) {
6
7 // Create Dependency
8 CustomerRepository repository =
9 new CustomerRepositoryImpl();
10
11 // Inject Dependency
12 CustomerService service =
13 new CustomerService(repository);
14
15 // Use Service
16 service.getCustomer(101);
17
18 service.getCustomer(200);
19 }
20 }
Problems Servers Terminal Data Source Explorer Properties Console
<terminated> DependencyInjectionTest [Java Application] C:\Users\divit\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\bin\javaw.exe (Jun 25, 2026, 7:03:22 PM – 7:03:22 PM elapsed)
Customer Name: Divith
Customer Name: Customer Not Found
Writable
Smart Insert
20 : 2 [457]